HDOC DAY-46

C Programming Assignment: Bubble Sort and Occurrence of a Target

Aim

To write a C program that sorts a variable-length integer array in ascending order using Bubble Sort and pointers, then finds the first and last indices of a target value in the sorted array.

Algorithm

1. Read the number of elements, n, and the n integer values.
2. Pass the array and its size to Bubble_sort() using a pointer.
3. Compare adjacent elements and swap them when they are in the wrong order.
4. Display the sorted array.
5. Call occurrence() and read the target integer.
6. Scan the sorted array to record the first and last indices where the target occurs.
7. If the target is absent, display -1, -1; otherwise display the target and both indices.

C Program

```
#include <stdio.h>

#define MAX 100

void Bubble_sort(int *arr, int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *arr, int n)
{
    int target, i;
    int first = -1, last = -1;

    printf("Enter target: ");
    scanf("%d", &target);

    for (i = 0; i < n; i++)
    {
        if (*(arr + i) == target)
        {
            if (first == -1)
                first = i;
            last = i;
        }
    }

    if (first == -1)
    {
        printf("-1, -1\\n");
    }
    else
    {
```

```
        printf("First occurrence: %d at index %d\\n", target, first);
        printf("Last occurrence: %d at index %d\\n", target, last);
    }
}

int main(void)
{
    int nums[MAX], n, i;

    printf("Enter array size (1-%d): ", MAX);
    if (scanf("%d", &n) != 1 || n < 1 || n > MAX)
    {
        printf("Invalid array size.\\n");
        return 1;
    }

    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    Bubble_sort(nums, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\\n");

    occurrence(nums, n);

    return 0;
}
```

Sample Output 1 — Target Occurs More Than Once

```
Enter array size (1-100): 6
Enter 6 integers: 7 5 8 7 10 8
Sorted array: 5 7 7 8 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 4
```

Sample Output 2 — Target Occurs Once

```
Enter array size (1-100): 5
Enter 5 integers: 5 7 7 8 10
Sorted array: 5 7 7 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 3
```

Sample Output 3 — Target Not Present

```
Enter array size (1-100): 6
Enter 6 integers: 5 7 7 8 8 10
Sorted array: 5 7 7 8 8 10
Enter target: 6
-1, -1
```

Result

The program sorts the input array in ascending order using Bubble Sort and pointers. It then displays the first and last indices of the target value, or -1, -1 when the target is not present.